Lec 4B: Virtualization & Cloud computing

Graduate Systems (CSE638)

Instructor: Rinku Shah

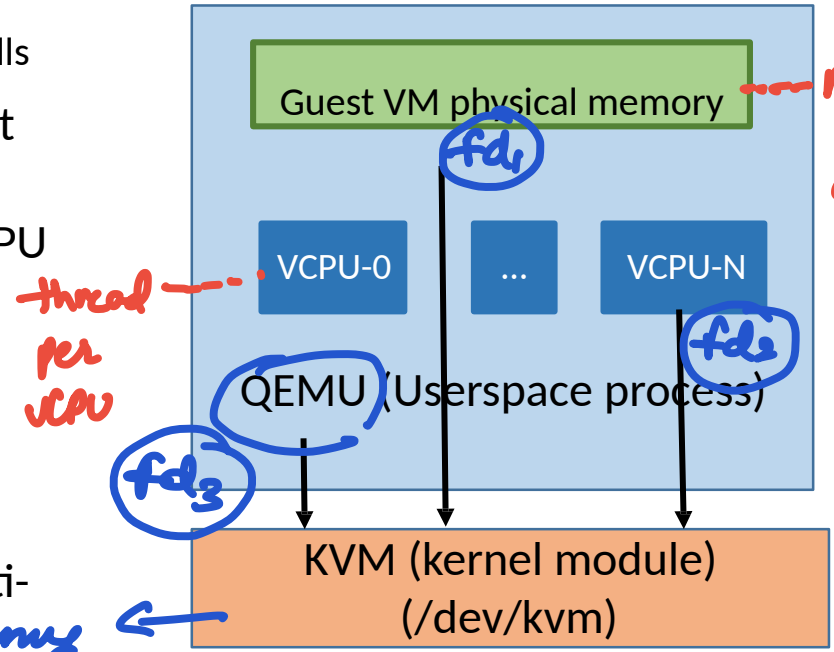
Content taken from:
- Design and Engineering of Computer Systems, Mythili Vutukuru (IITB)
https://www.cse.iitb.ac.in/~mythili/decs/
- CS 695: Virtualization and Cloud Computing, Mythili Vutukuru (IITB)
https://www.cse.iitb.ac.in/~mythili/virtcc/



INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY
DELHI

QEMU architecture

- QEMU is userspace process
- KVM exposes a dummy device
  - QEMU talks to KVM via open/ioctl syscalls
- Allocates memory via mmap for guest VM physical memory
- Creates one thread for each virtual CPU (VCPU) in guest
- Multiple file descriptors to /dev/kvm (one for QEMU, one for VM, one for VCPU and so on)
  - ioctl on fds to talk to KVM
- Host OS sees QEMU as a regular multi-threaded process



QEMU operation

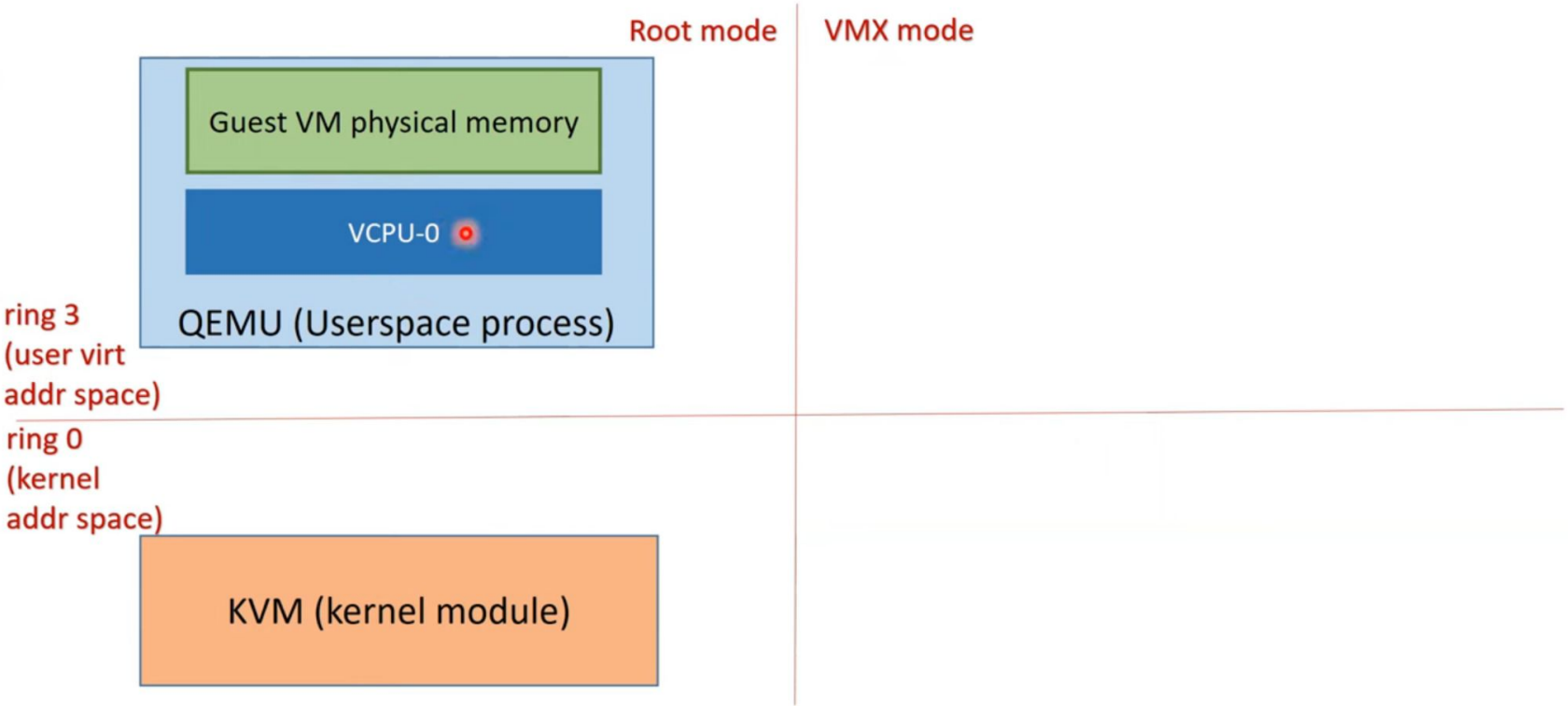
```
open(/dev/kvm)
ioctl(qemu_fd, KVM_CREATE_VM)
ioctl(vm_fd, KVM_CREATE_VCPU)

for(;;) { //each VCPU runs this loop
  ioctl(vcpu_fd, KVM_RUN)
  switch(exit_reason) {
    case KVM_EXIT_IO: //do I/O case
    KVM_EXIT_HLT:
  }
}
```

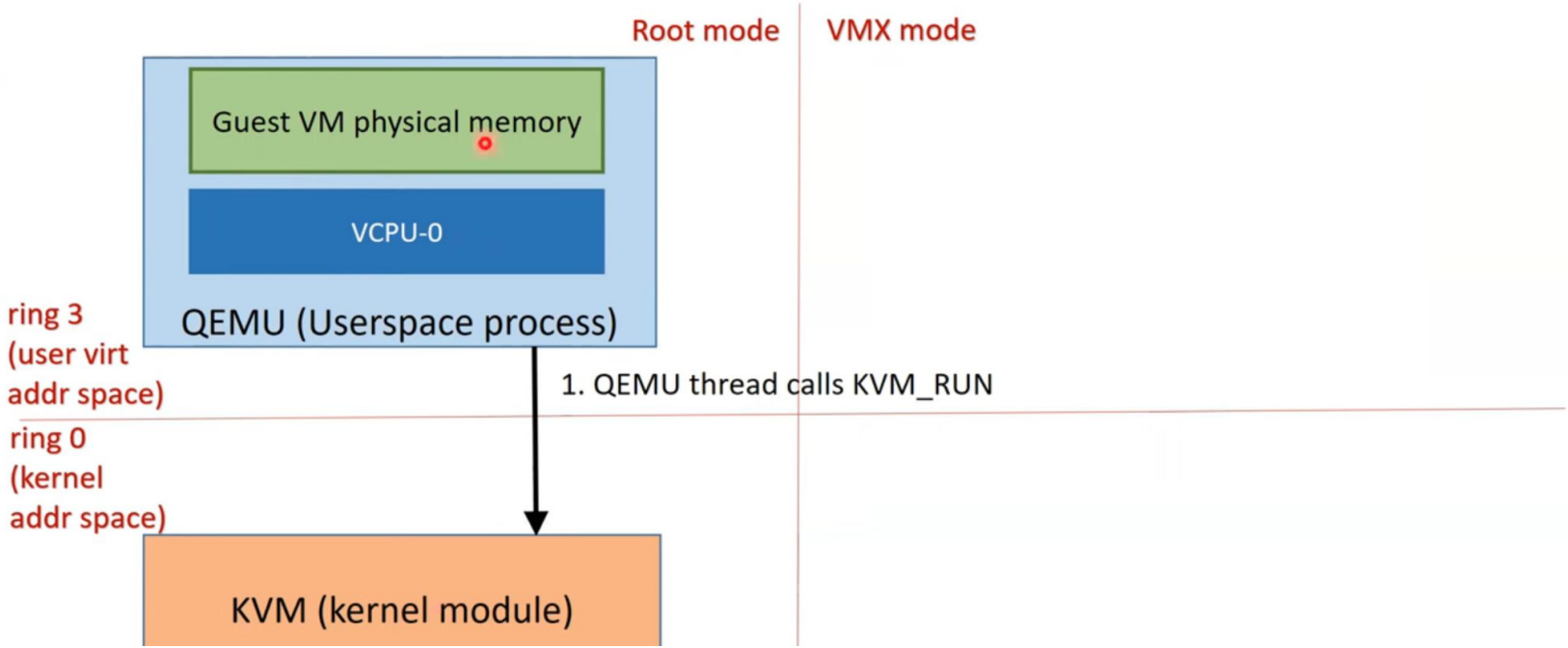
This ioctl system call blocks this thread, KVM switches to VMX mode, runs guest VM

- Returns to QEMU on host when VM exits from VMX mode.
- QEMU handles exit and returns to guest VM

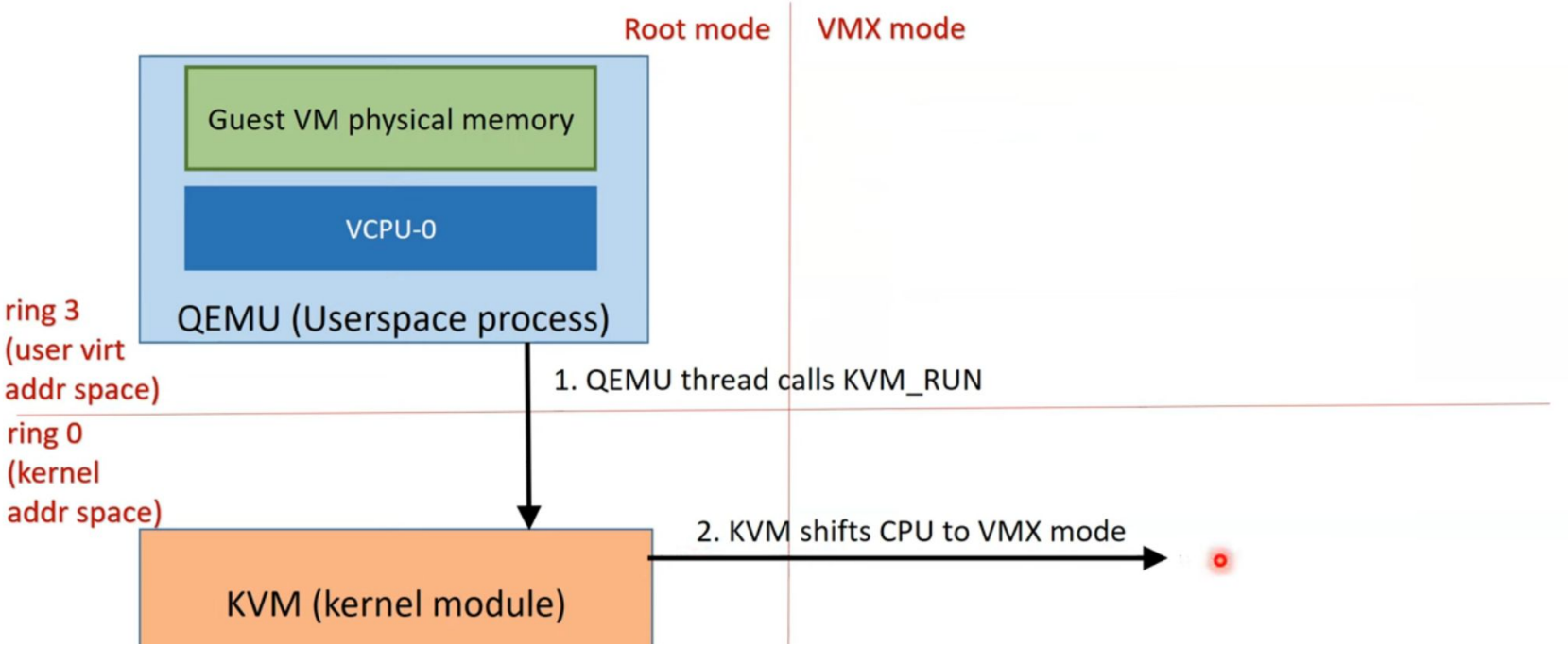
QEMU/KVM operation



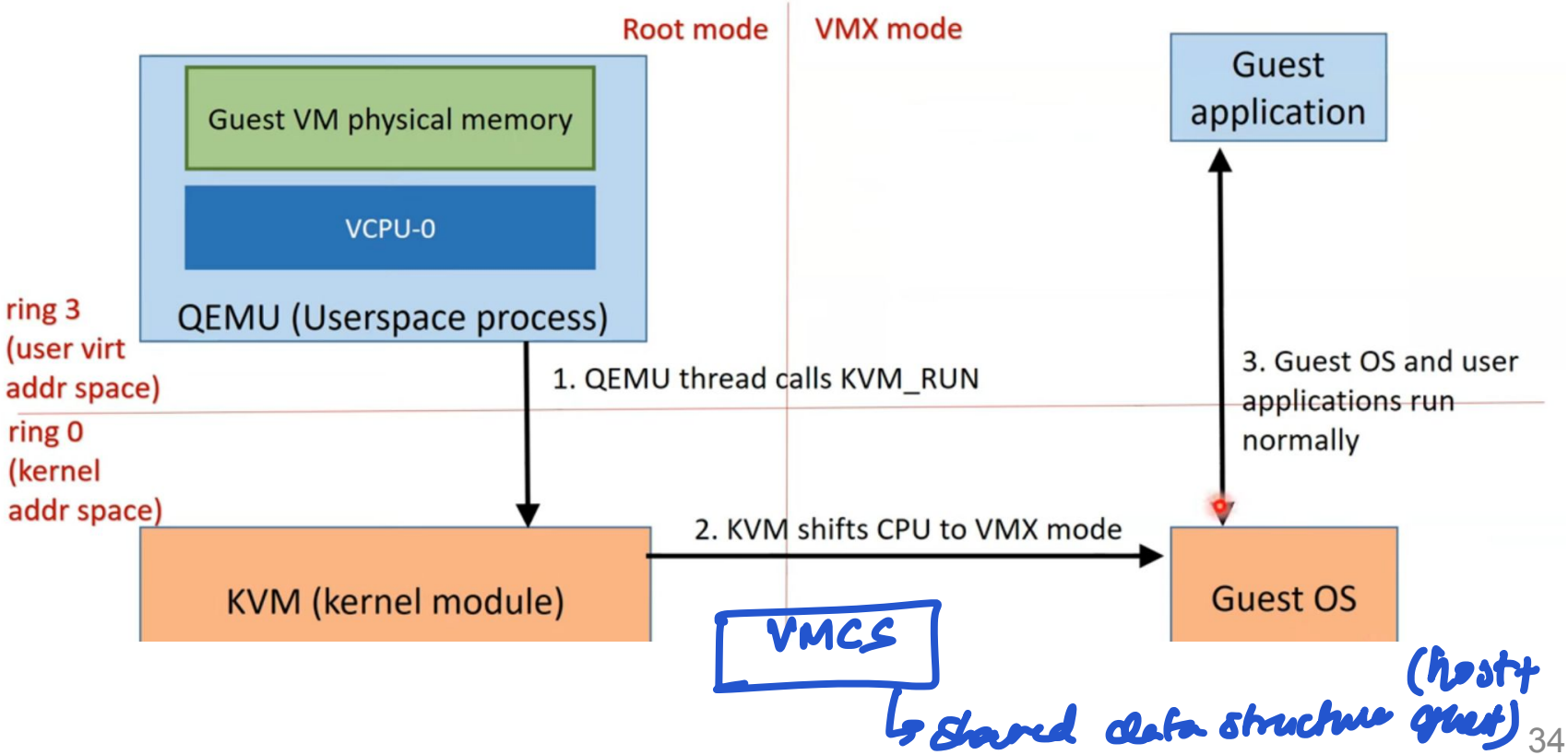
QEMU/KVM operation



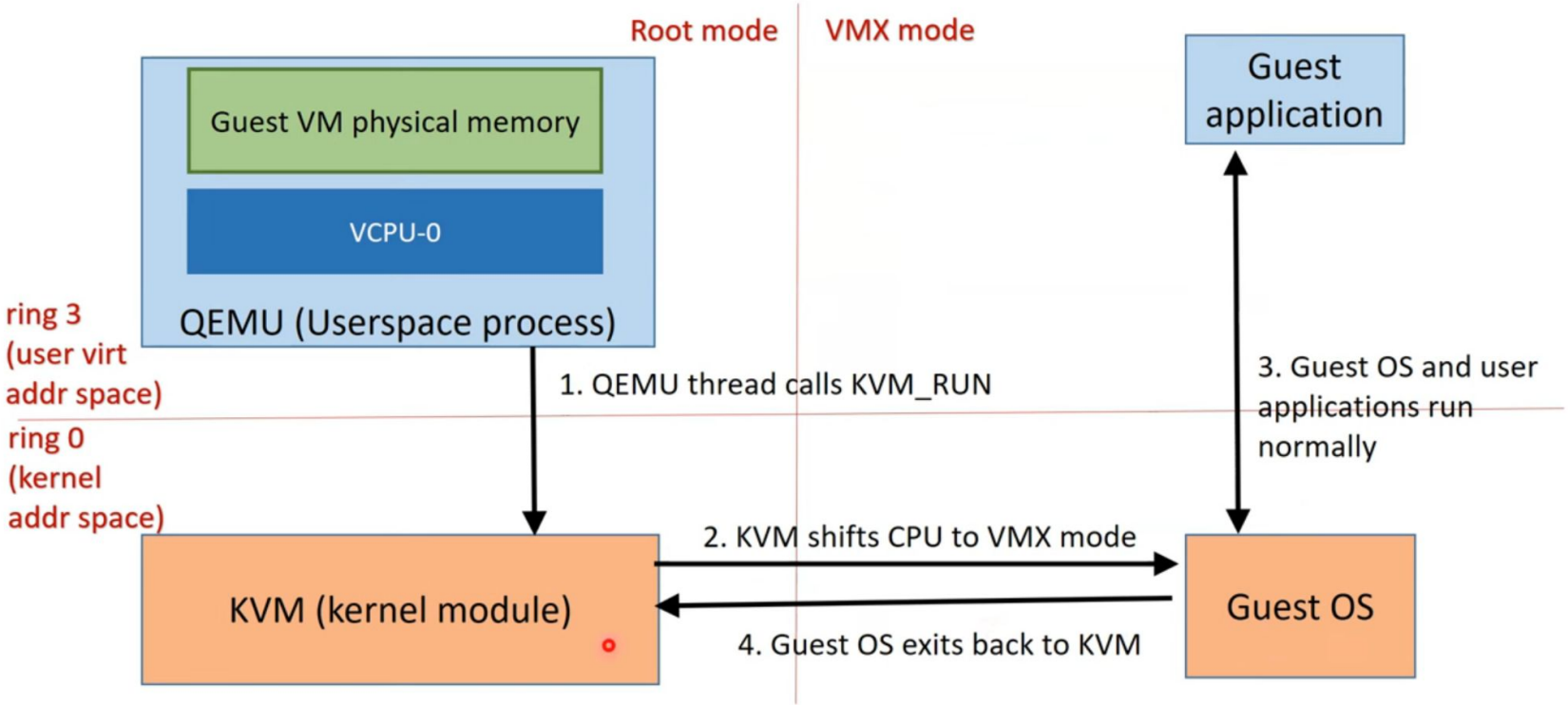
QEMU/KVM operation



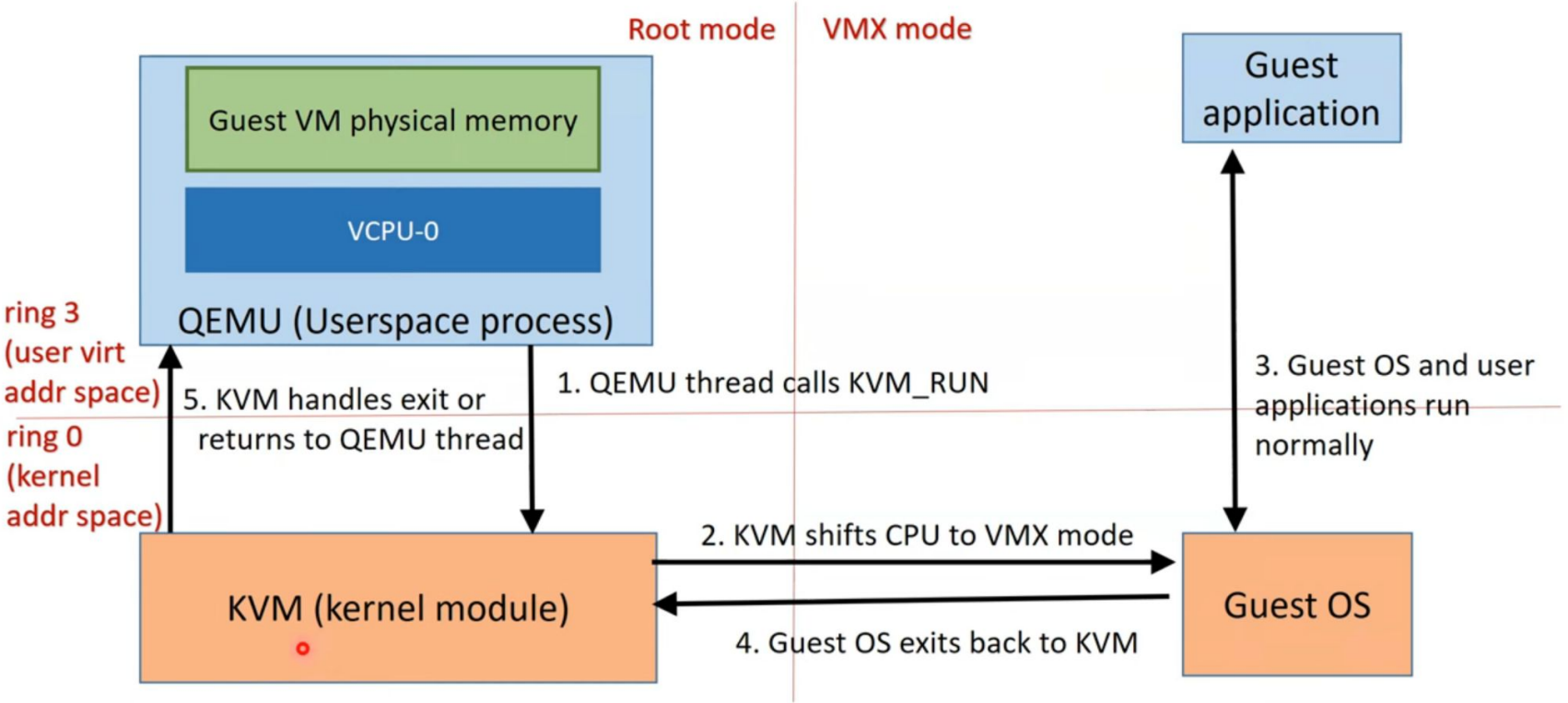
QEMU/KVM operation



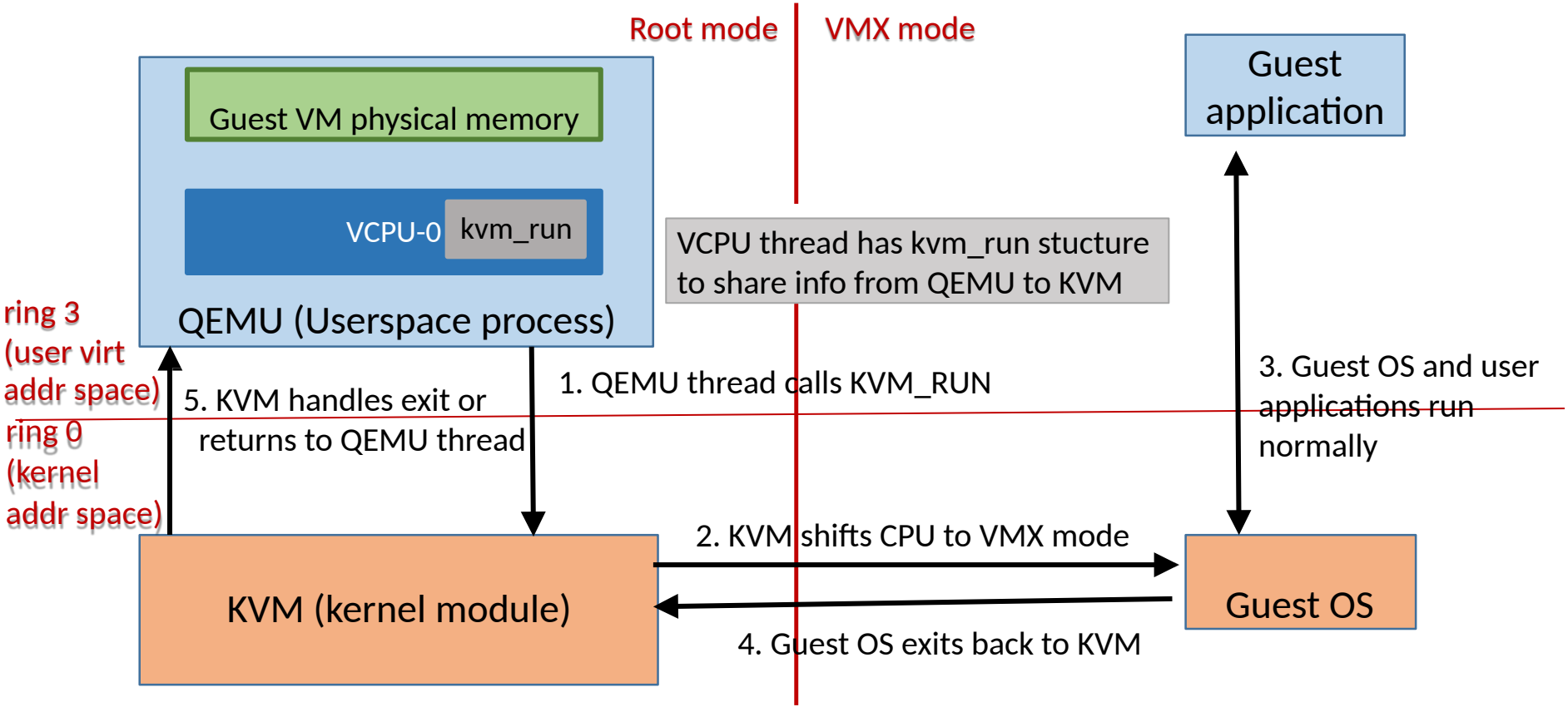
QEMU/KVM operation



QEMU/KVM operation



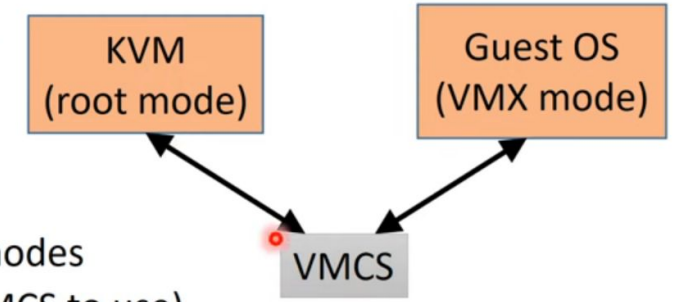
QEMU/KVM operation



VMX mode

- Special CPU instructions to enter and exit VMX mode
  - VMLAUNCH, VMRESUME invoked by KVM to enter VMX mode
  - VMEXIT invoked by guest OS to exit VMX mode
- On VMX entry/exit instructions, CPU switches context between host OS to guest OS
  - Page tables (address space), CPU register values etc switched
  - Hardware manages the mode switch
- Where is CPU context stored during mode switch?
  - Cannot be stored in host OS or guest OS data structures alone (why?)
  - VMCS (VM control structure), also called VMCB (VM control block)

VM control structure (VMCS)

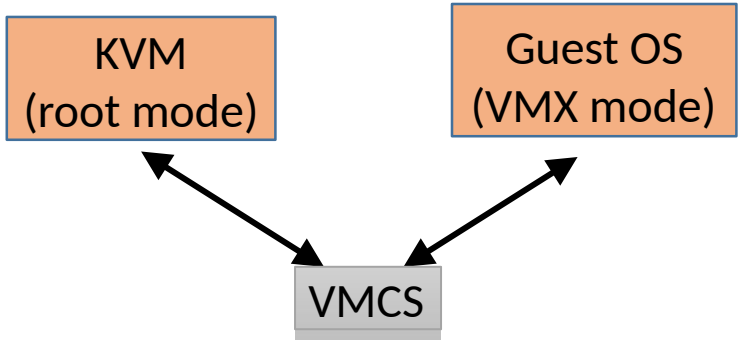


- What is VMCS?
  - Common memory area accessible in both modes
  - One VMCS per VM (KVM tells CPU which VMCS to use)
- What is stored in VMCS?
  - Host CPU context: Stored when launching VM, restored on VM exit
  - Guest CPU context: Stored on VM exit, restored when VM is run
  - Guest entry/execution/exit control area: KVM can configure guest memory and CPU context, which instructions and events should cause VM to exit
  - Exit information: Exit reason and any other exit-related information
- VMCS information (e.g., exit reason) exchanged with QEMU via kvm_run structure
  - VMCS only accessible to KVM in kernel mode, not to QEMU userspace

VMX mode

- Special CPU instructions to enter and exit VMX mode
  - VMLAUNCH, VMRESUME invoked by KVM to enter VMX mode
  - VMEXIT invoked by guest OS to exit VMX mode
- On VMX entry/exit instructions, CPU switches context between host OS to guest OS
  - Page tables (address space), CPU register values etc switched
  - Hardware manages the mode switch
- Where is CPU context stored during mode switch?
  - Cannot be stored in host OS or guest OS data structures alone (why?)
  - VMCS (VM control structure), also called VMCB (VM control block)

VM control structure (VMCS)

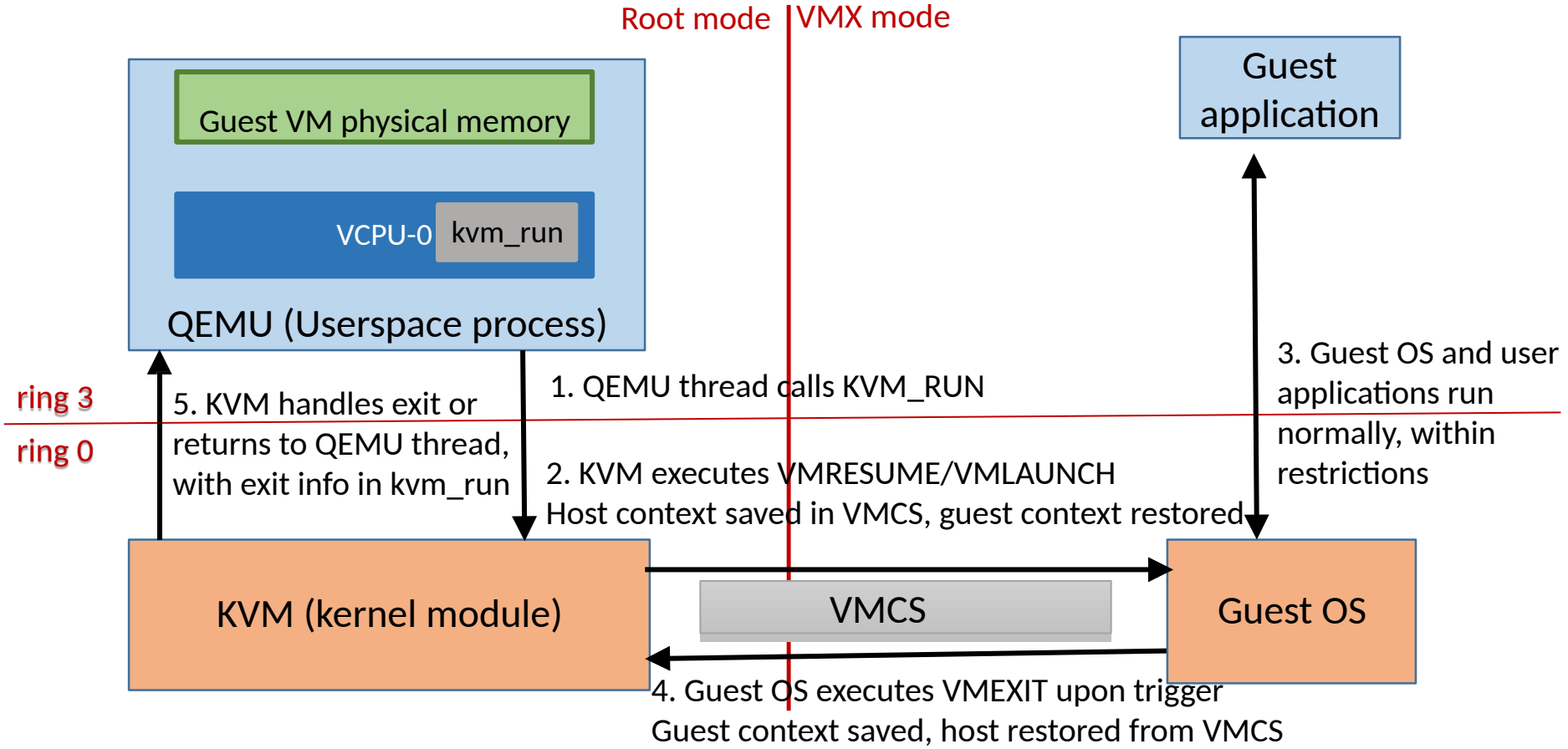


- What is VMCS?
  - Common memory area accessible in both modes
  - **One VMCS per VM** (KVM tells CPU which VMCS to use)
- What is stored in VMCS?
  - Host CPU context: Stored when launching VM, restored on VM exit
  - Guest CPU context: Stored on VM exit, restored when VM is run
  - Guest entry/execution/exit control area: KVM can configure guest memory and CPU context, which instructions and events should cause VM to exit
  - Exit information: Exit reason and any other exit-related information
- VMCS information (e.g., exit reason) exchanged with QEMU via kvm_run structure
  - VMCS only accessible to KVM in kernel mode, not to QEMU userspace

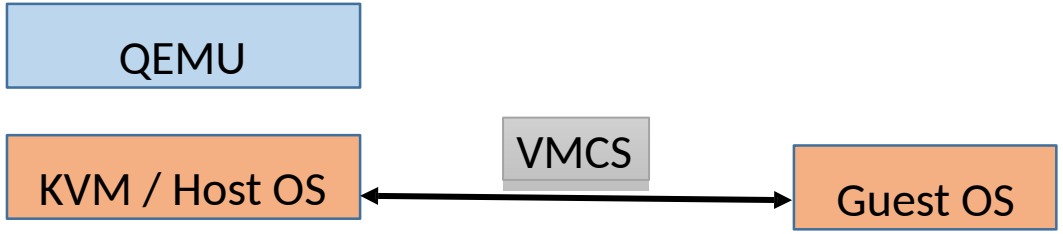
VMX mode execution

- **How is guest OS execution in VMX mode different?**
- Restrictions on guest OS execution, **configurable exits to KVM**
  - Guest OS exits to KVM on certain instructions (e.g., I/O device access)
- **No hardware access to guest, emulated by KVM**
  - Guest OS usually exits on interrupts (interrupts handled by KVM, assigned to the appropriate host or guest OS)
  - KVM can **inject virtual interrupts** to guest OS during VMX mode entry
- All of the above controlled by KVM via VMCS
- Mimics the **trap-and-emulate architecture** with hardware support
  - Guest runs in a (special) ring 0, but trap-and-emulate achieved

QEMU/KVM operation revisited

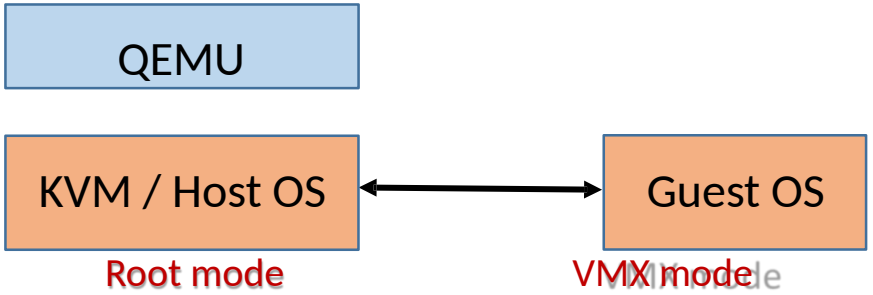


Host view



- Host sees QEMU as regular multithreaded process
  - Process that has memory-mapped memory, talks to KVM device via ioctl calls
  - Multiple QEMU VCPU threads can be scheduled in parallel on multiple cores
- When KVM launches a VM, host OS context is stored in VMCS
  - Host OS execution is suspended (all host processes stop)
  - CPU loads guest OS context and guest OS starts running
- When guest OS exits, host OS context is restored from VMCS
  - Host OS resumes in KVM, where it stopped execution
  - KVM can return to QEMU, or host can switch to another process
  - Host OS is not aware of guest OS execution

Summary



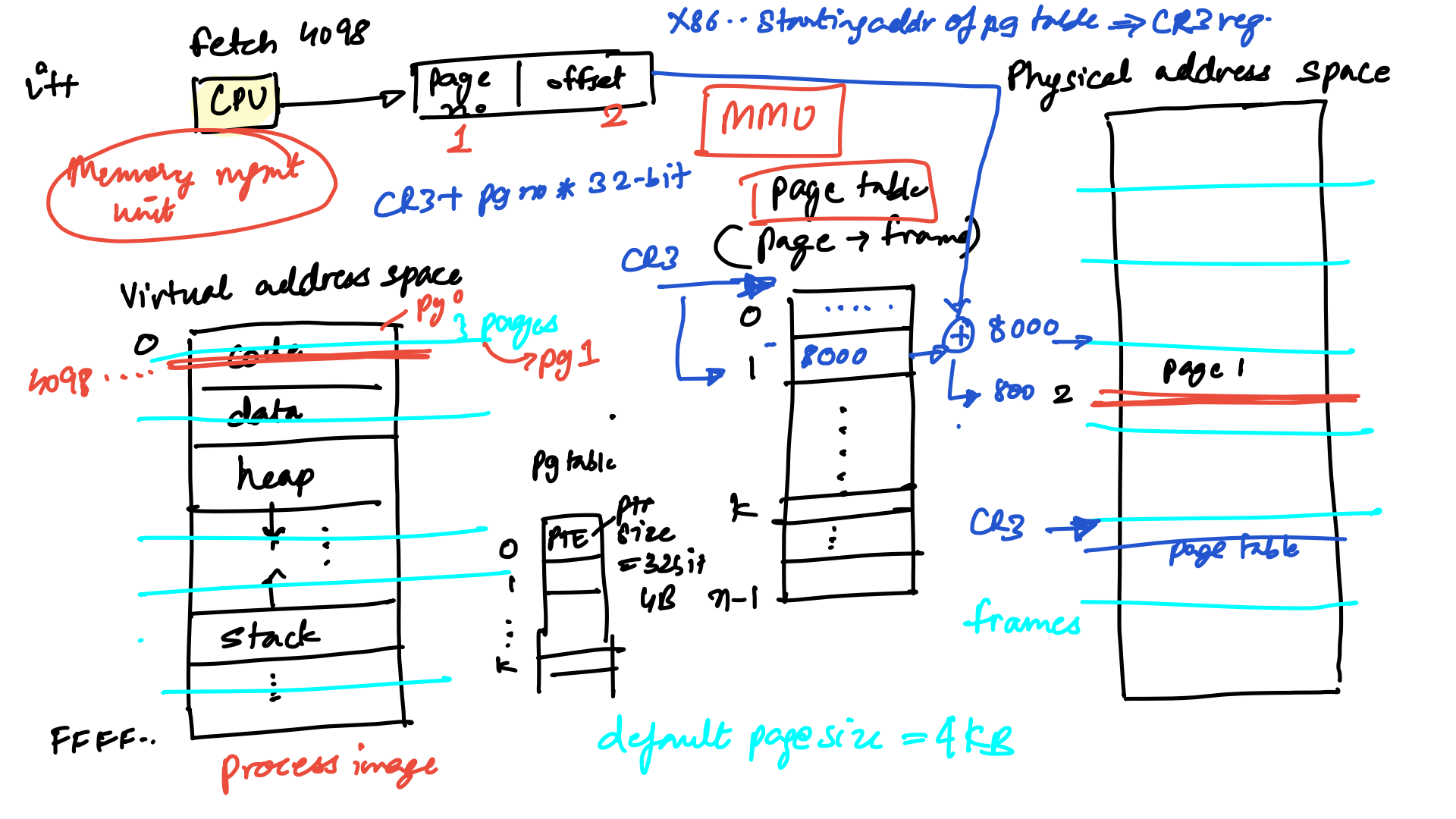
- Hardware-assisted CPU virtualization in QEMU/KVM
  - QEMU creates guest physical memory, one thread per VPCU
  - QEMU VCPU thread gives KVM_RUN command to KVM kernel module
  - KVM configures VM information in VMCS, launches guest OS in VMX mode
  - Guest OS runs natively on CPU until VM exit happens
  - Control returns to KVM/Host OS on VM exit
  - VM exits handled by KVM or QEMU
  - Host schedules QEMU like any other process, not aware of guest OS

Memory Virtualization

Memory address translation in Processes

Virtual memory

- Addresses assigned in memory image, that are used by CPU to load and store are virtual/logical addresses
  - These virtual addresses are not actual addresses occupied by instructions/data of process, but assigned from 0 for convenience
- Actual addresses where instructions/data bytes of process are stored are called physical addresses
  - RAM hardware needs physical addresses to fetch bytes
- CPU requests code/data at virtual addresses
  - Translated to physical address so that RAM can fetch it
- Memory is allocated at granularity of pages (usually 4KB)
  - Logical pages of a process are stored in physical frames in memory
  - Logical page numbers translated to physical frame numbers


fetch 4098
CPU
Page
offset
1
2
MMU
Memory mgmt unit
CR3+ pg no * 32-bit
X86 .. Starting addr of pg table ⇒ CR3 reg.
Physical address space
Page table
Page → frame
CR3
0
1
8000
k
n-1
8000
800 2
Page 1
CR3
page table
frames
Virtual address space
pg 0
3 pages
pg 1
4098 ....
0
code
data
heap
stack
FFFF..
Process image
Pg table
PTE
Ptr
Size
=32bit
4B
0
1
k
default page size = 4KB

pg table size ⇒ no. of pages * ptr size

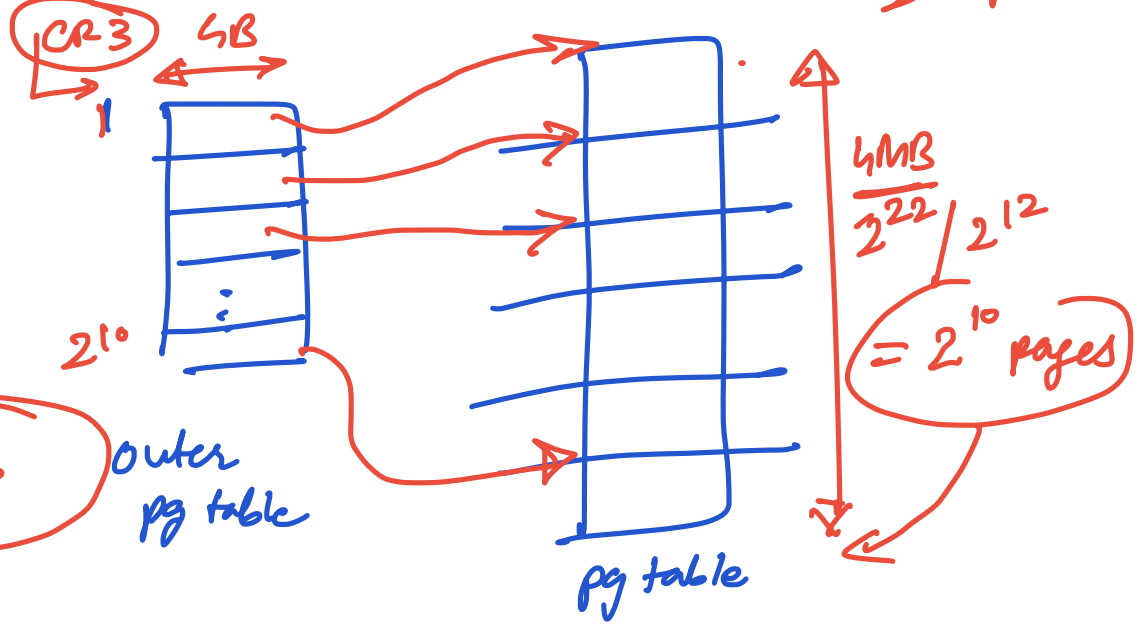
32-bit m/c. = 2^{20} * 32 = 4MB

max # of pages in a 32-bit m/c = $2^{32}/2^{12} = 2^{20}$

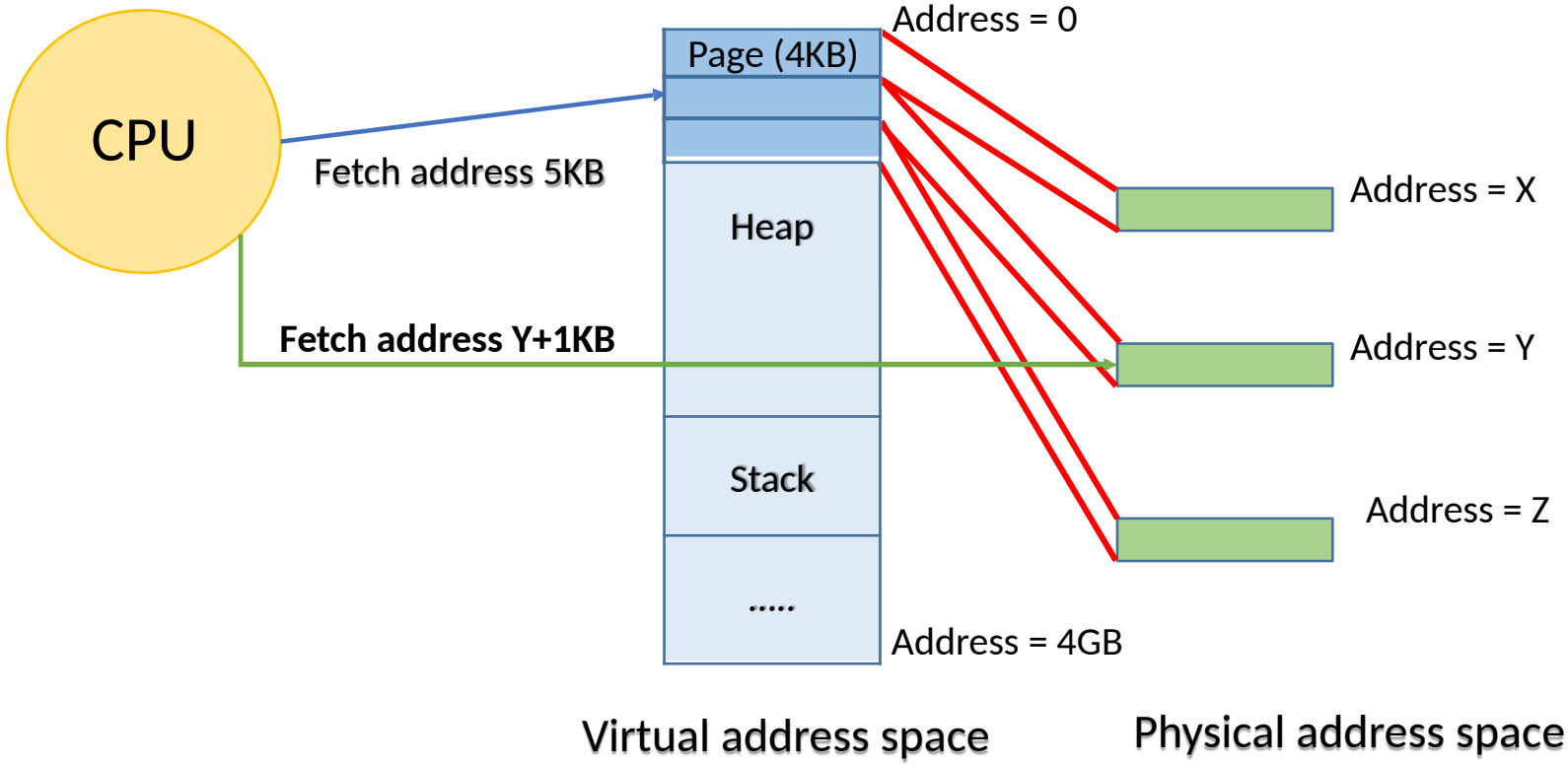
pg size = 4KB = 2^{12}



→ Two-level page table

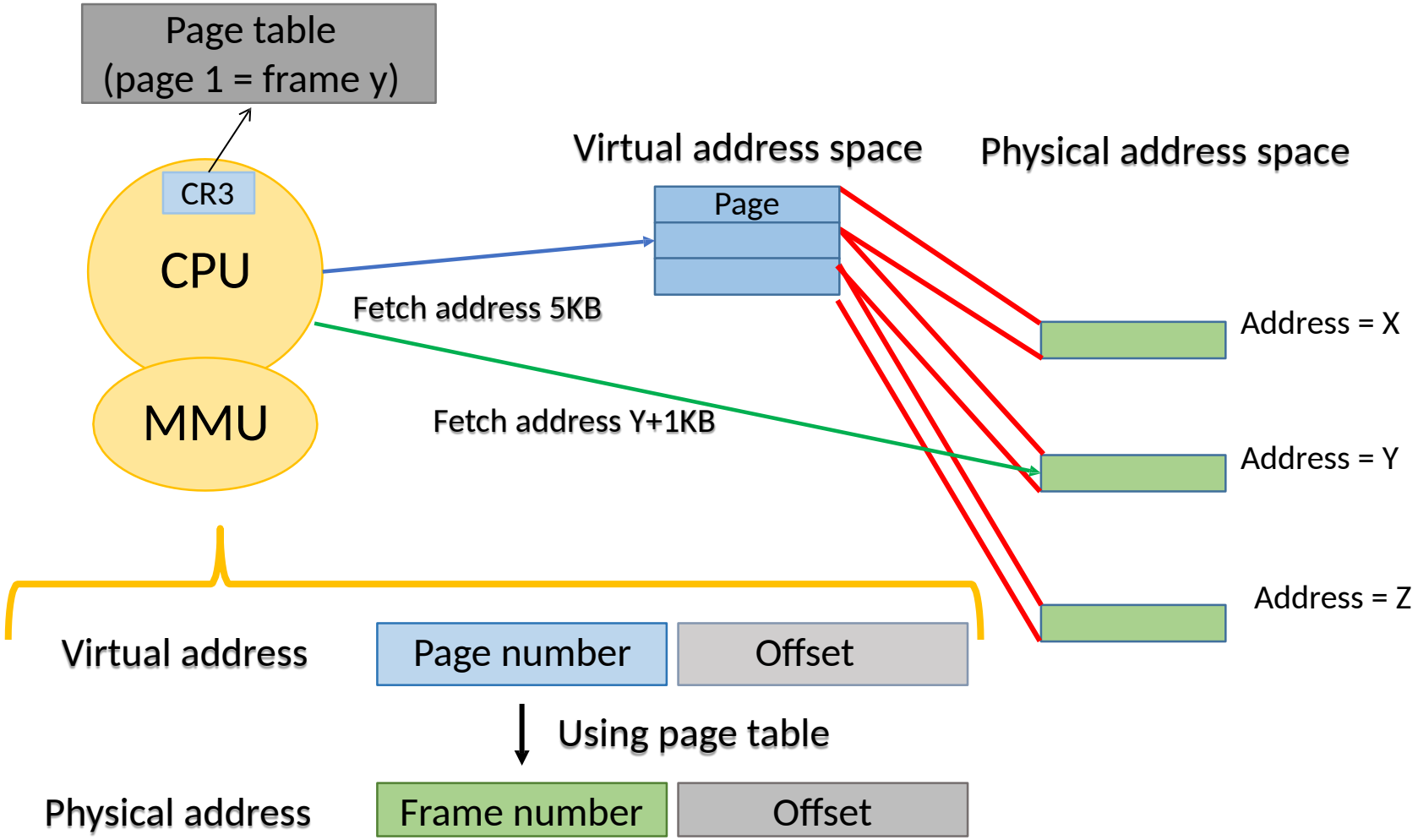


Virtual and physical address spaces



Address translation with paging

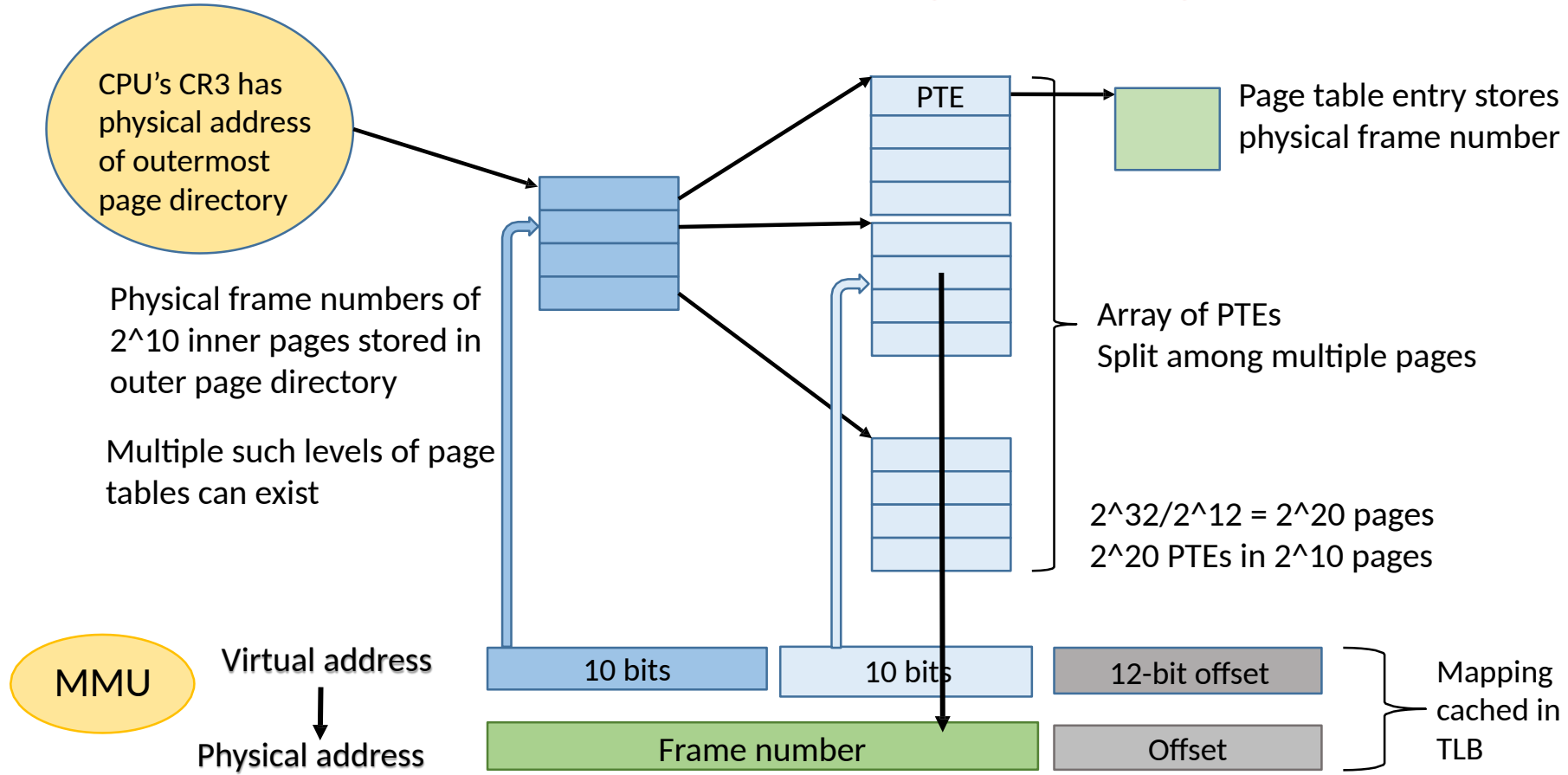
- On every memory access, a piece of hardware called MMU (memory management unit) translates virtual addresses to physical addresses
- Page table of a process stores the mapping of logical page number → physical frame number
  - OS builds the page table when allocation memory
  - MMU uses this page table to translate addresses
- MMU looks up CR3 register of CPU (x86) which stores location of page table of current process
  - Looks up page number in page table to translate address
  - CR3 reset on every context switch
- Recent address translations cached in TLB (Translation Lookaside Buffer) located within MMU

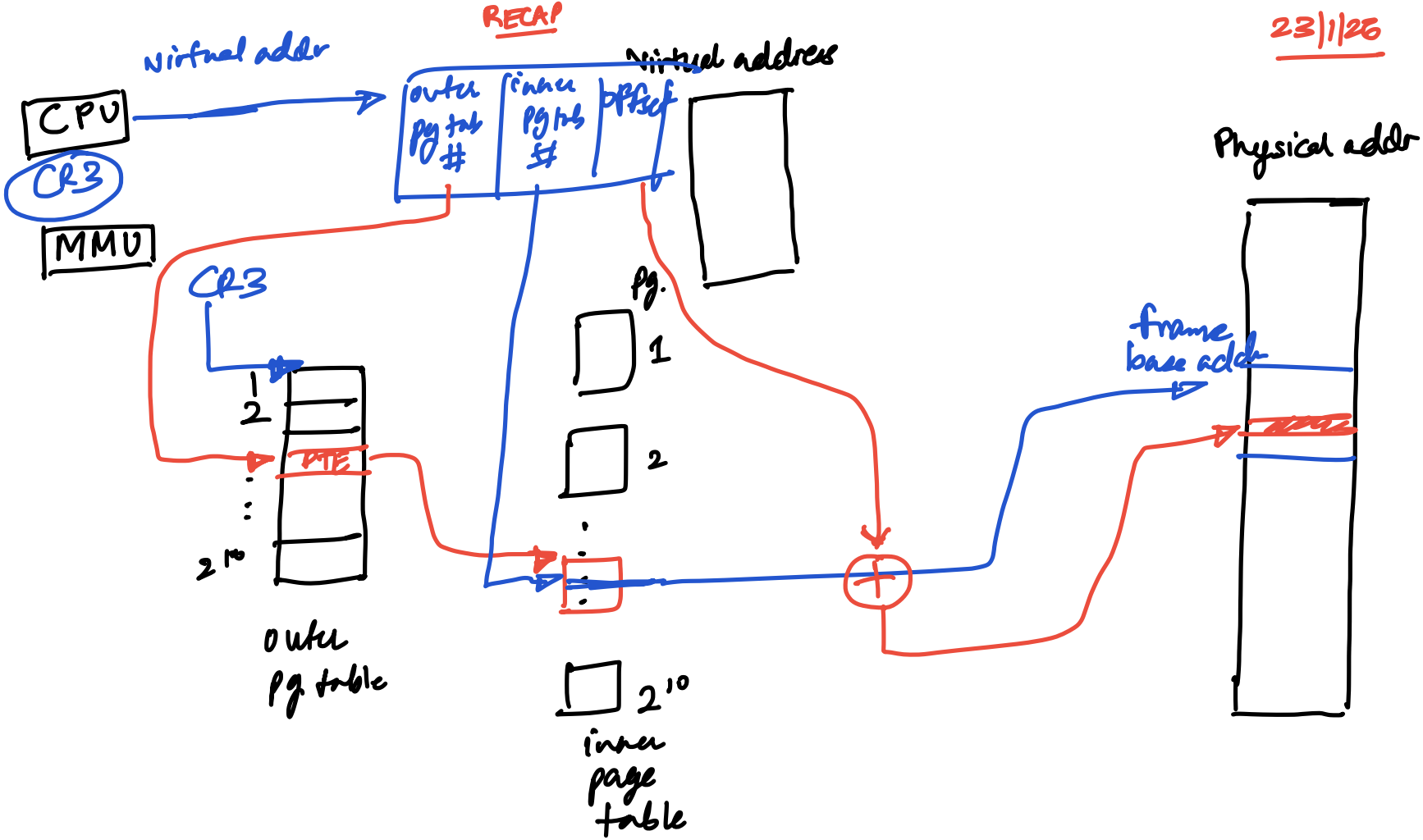

Page table
(page 1 = frame y)
CR3
CPU
MMU
Virtual address space
Physical address space
Page
Fetch address 5KB
Fetch address Y+1KB
Address = X
Address = Y
Address = Z
Virtual address
Page number
Offset
Using page table
Physical address
Frame number
Offset

Hierarchical page tables

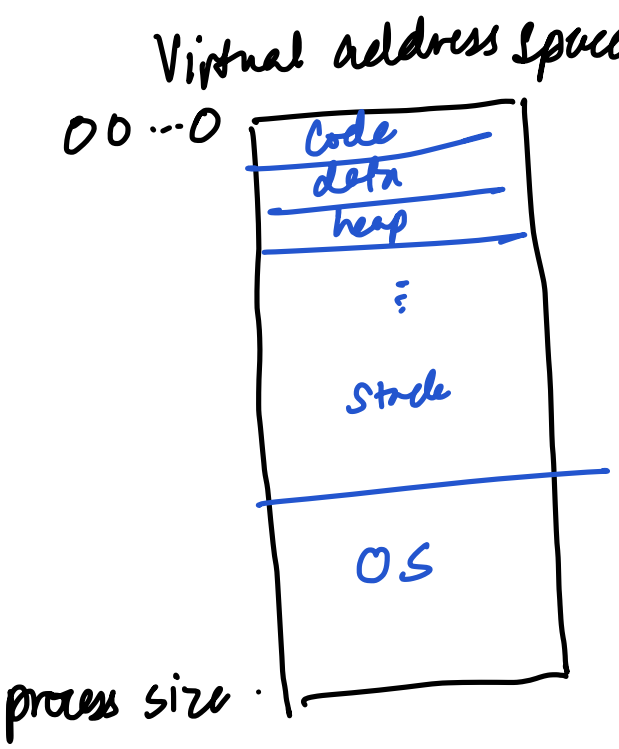
- Modern operating systems use hierarchical page tables
- 32-bit virtual address space (4GB), 2^12 byte (4KB) pages
  - Each process can have up to 2^32/2^12 = 2^20 pages
  - Each page has a page table entry (PTE), so 2^20 PTEs per process
  - Assuming each PTE is 4 bytes, all page table entries occupy 4*2^20 = 4MB
- Cannot store a large page table contiguously in memory, so page table of a process is stored in memory in page sized chunks
  - Each 4KB page stores 2^10 PTEs, so 2^20/2^10 = 2^10 pages to store all PTEs
  - Pointers to these “inner” pages stored in an outer page directory
  - In 32-bit architectures, one outer page can store physical frame numbers of all 2^10 inner page table pages, so 2-level page table
  - More levels in page table for 64-bit architectures

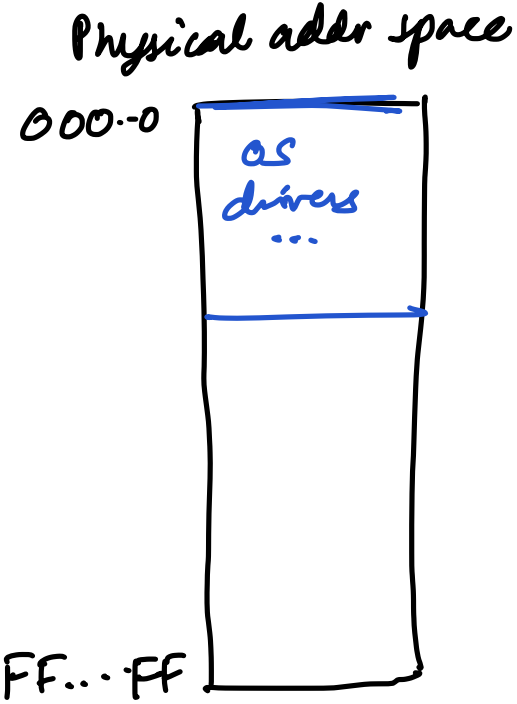
Page table lookup (walking the page table)



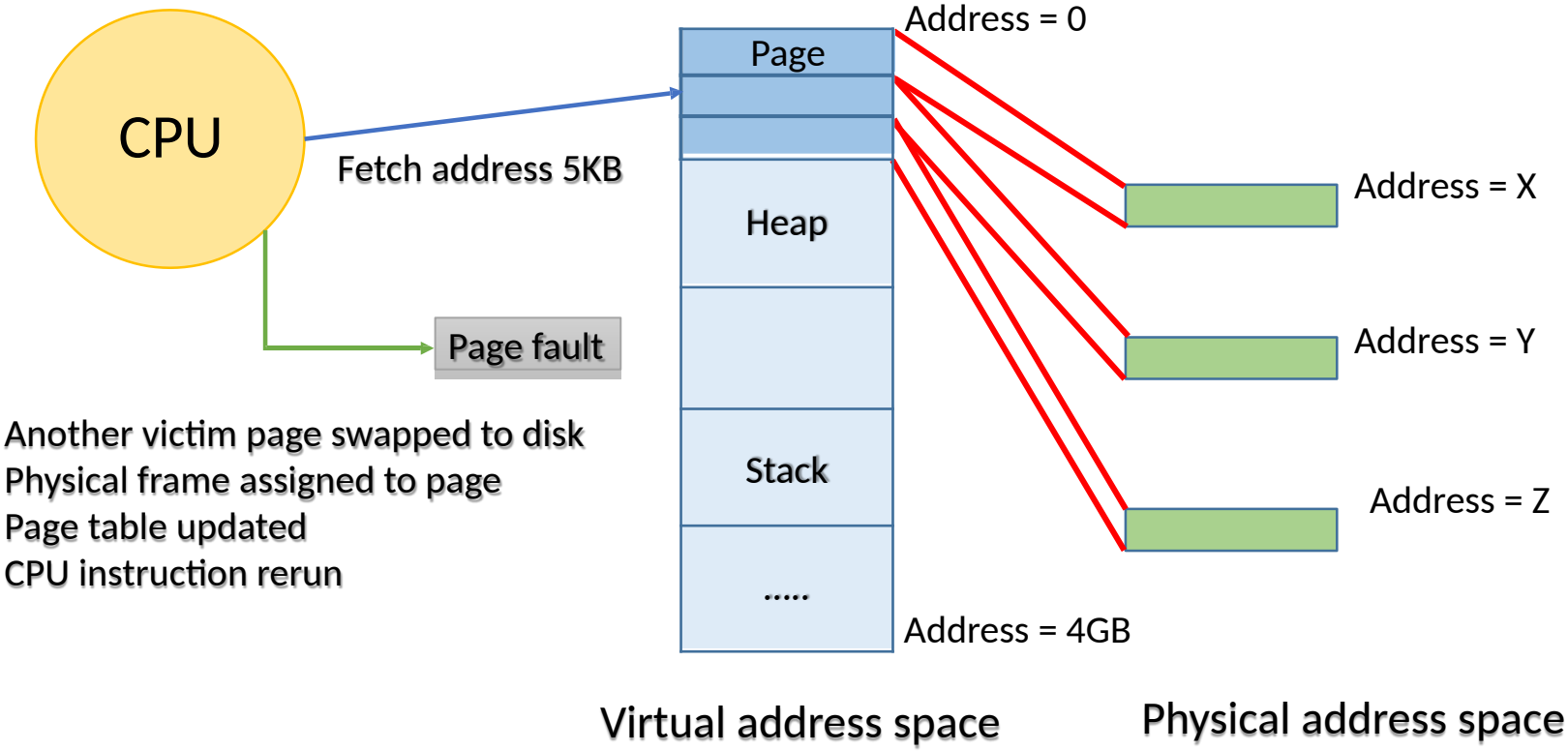

RECAP
23/1/26
Virtual addr
CPU
CR3
MMU
Virtual address
outer pg tab #
inner pg tab #
offset
Physical addr
CR3
pg.
1
2
PTE
2^{10}
frame base addr
outer pg table
2^{10}
inner page table

* Clarification about position of OS in the address space



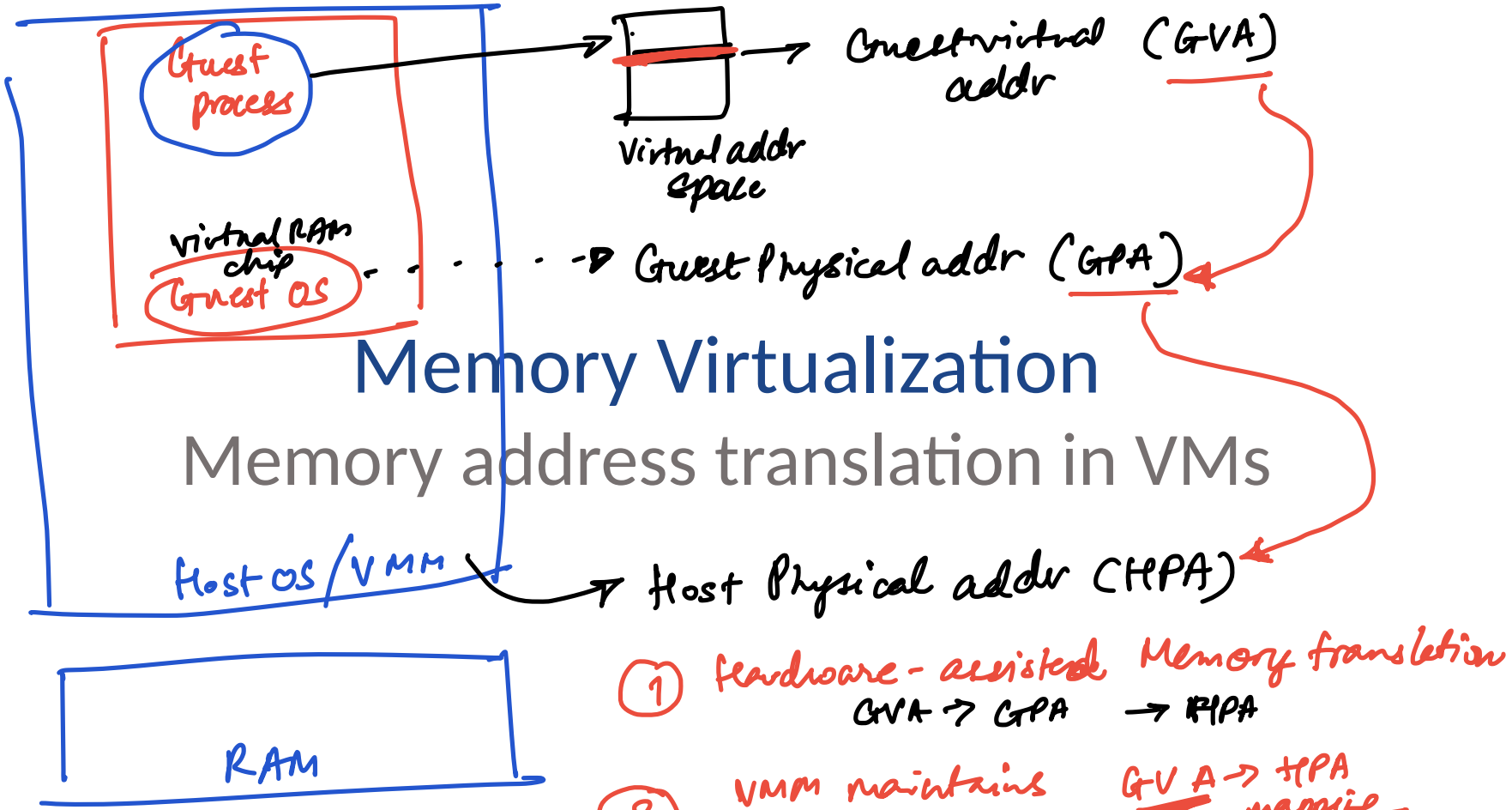


Demand Paging

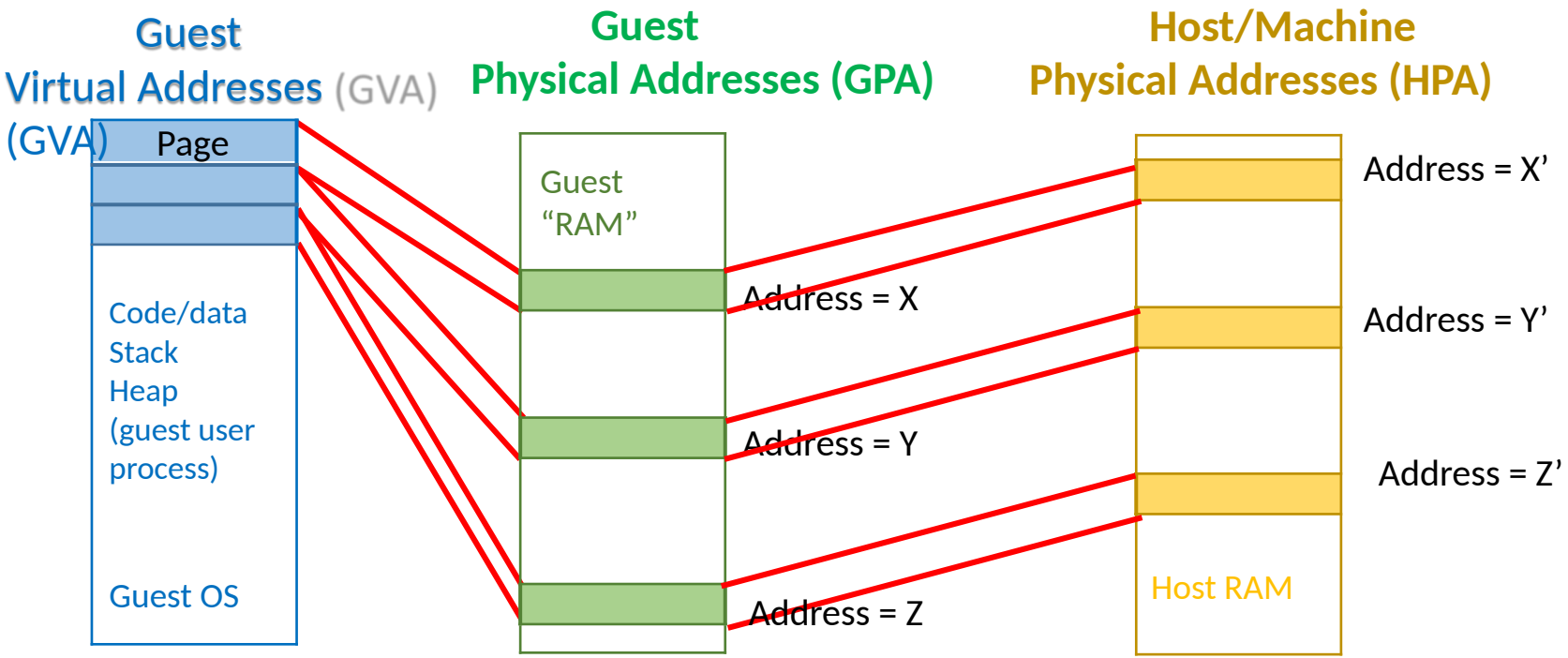


Memory Virtualization

Memory address translation in VMs



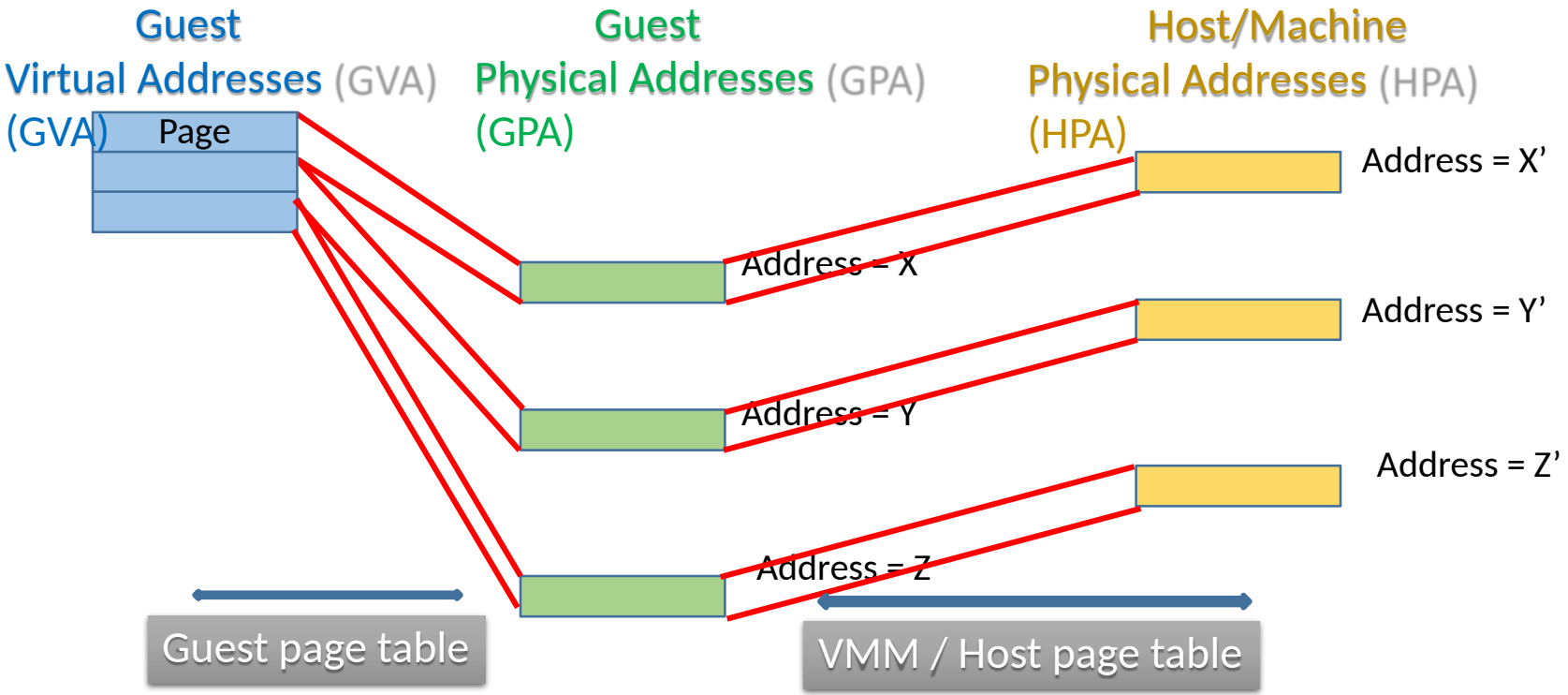
Memory virtualization problem



Guest "RAM" is actually memory of the userspace hypervisor process running on the host, which is mapped to host memory by the host's page table

Memory virtualization

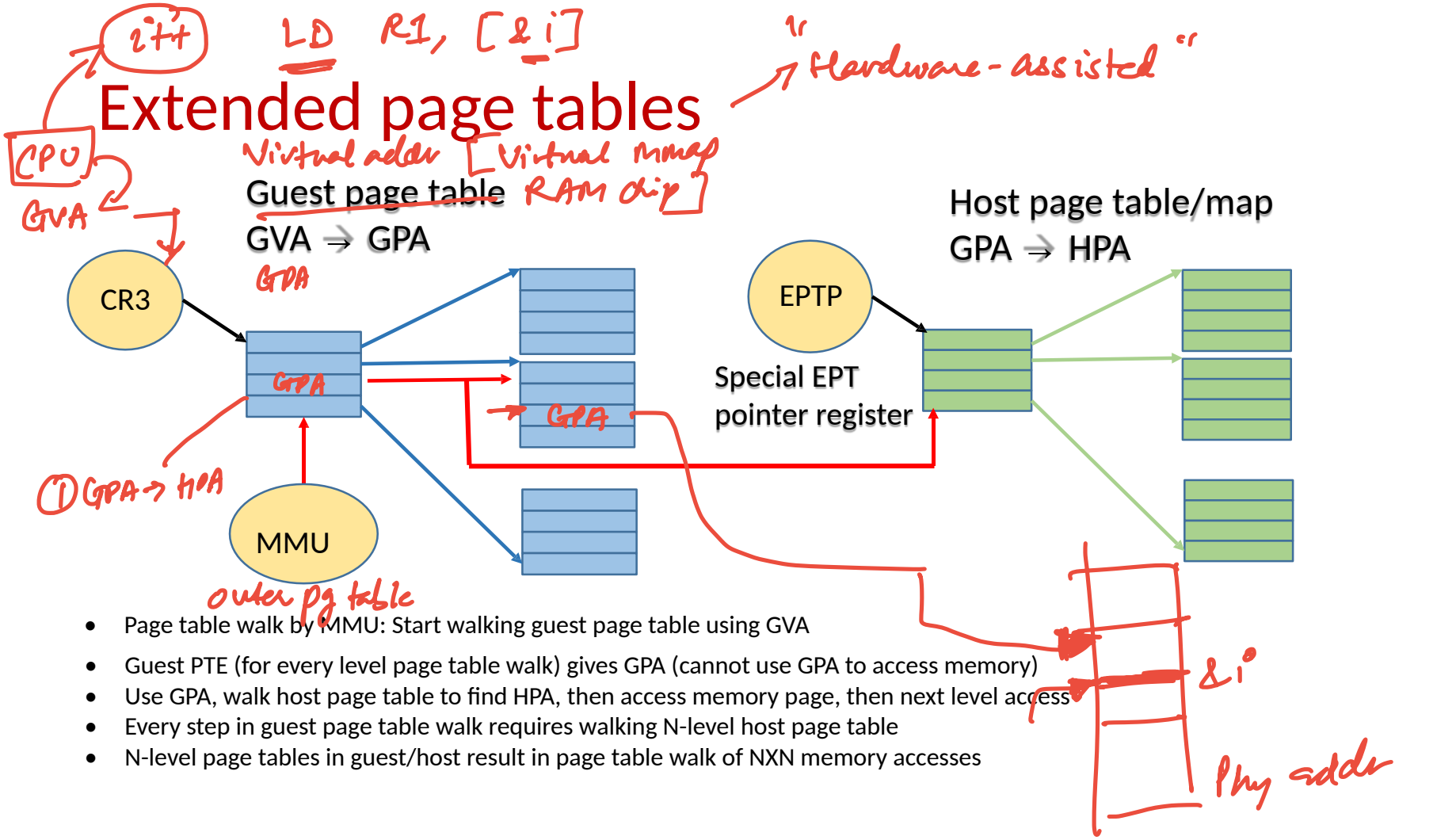
- What about address translation in virtual machines?



Techniques for memory virtualization

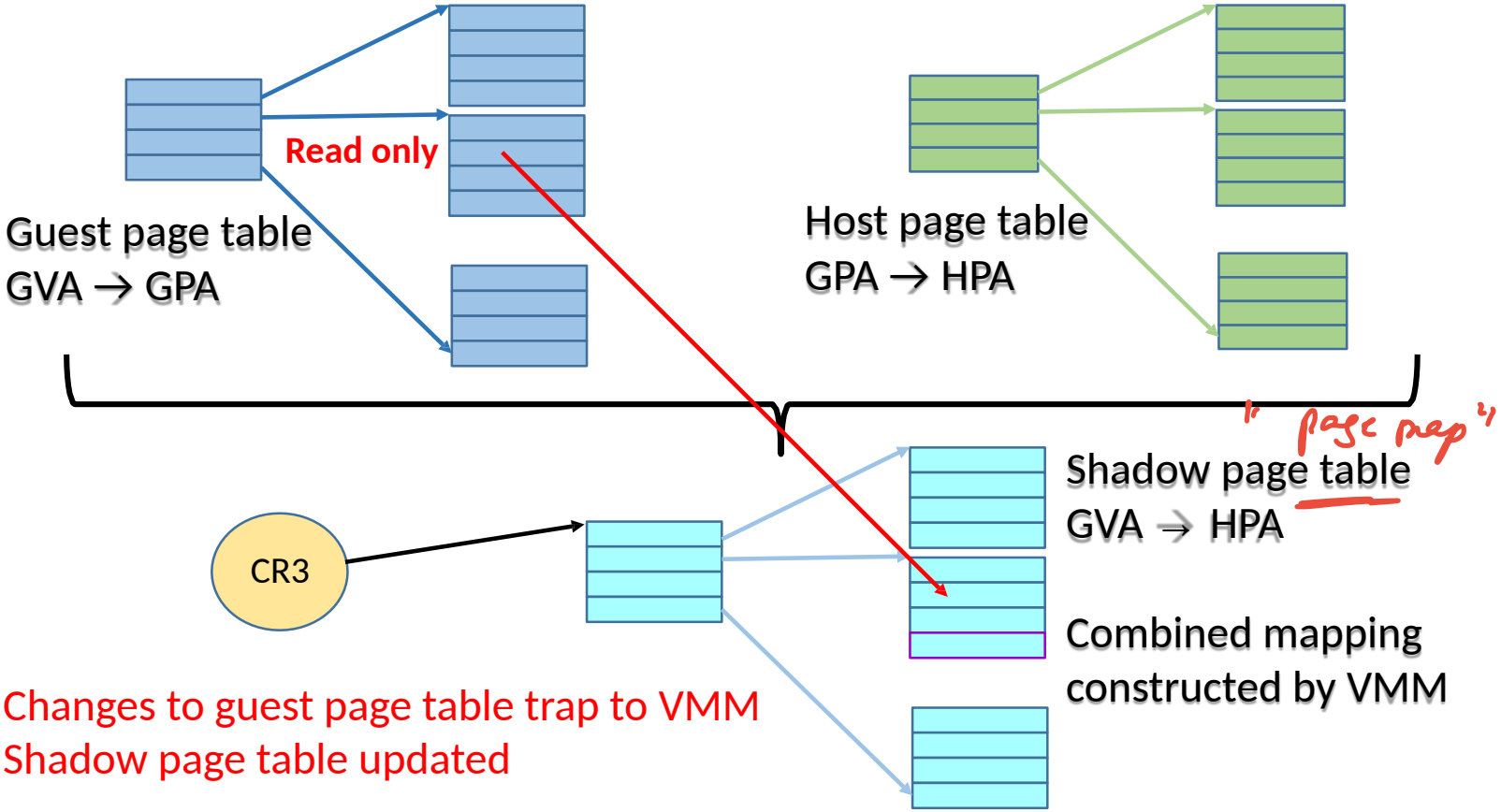
- Guest page table has GVA → GPA mapping
  - Each guest OS thinks it has access to all RAM starting at address 0
- VMM / Host OS has GPA → HPA mapping
  - Guest "RAM" pages are distributed across host memory
- Which page table should MMU use?
- Shadow paging: VMM creates a combined mapping GVA□HPA and MMU is given a pointer to this page table
  - VMM tracks changes to guest page table and updates shadow page table
- Extended page tables (EPT): MMU hardware is aware of virtualization, takes pointers to two separate page tables
  - Address translation walks both page tables
- EPT is more efficient but requires hardware support

Extended page tables



- Page table walk by MMU: Start walking guest page table using GVA
- Guest PTE (for every level page table walk) gives GPA (cannot use GPA to access memory)
- Use GPA, walk host page table to find HPA, then access memory page, then next level access
- Every step in guest page table walk requires walking N-level host page table
- N-level page tables in guest/host result in page table walk of NXN memory accesses

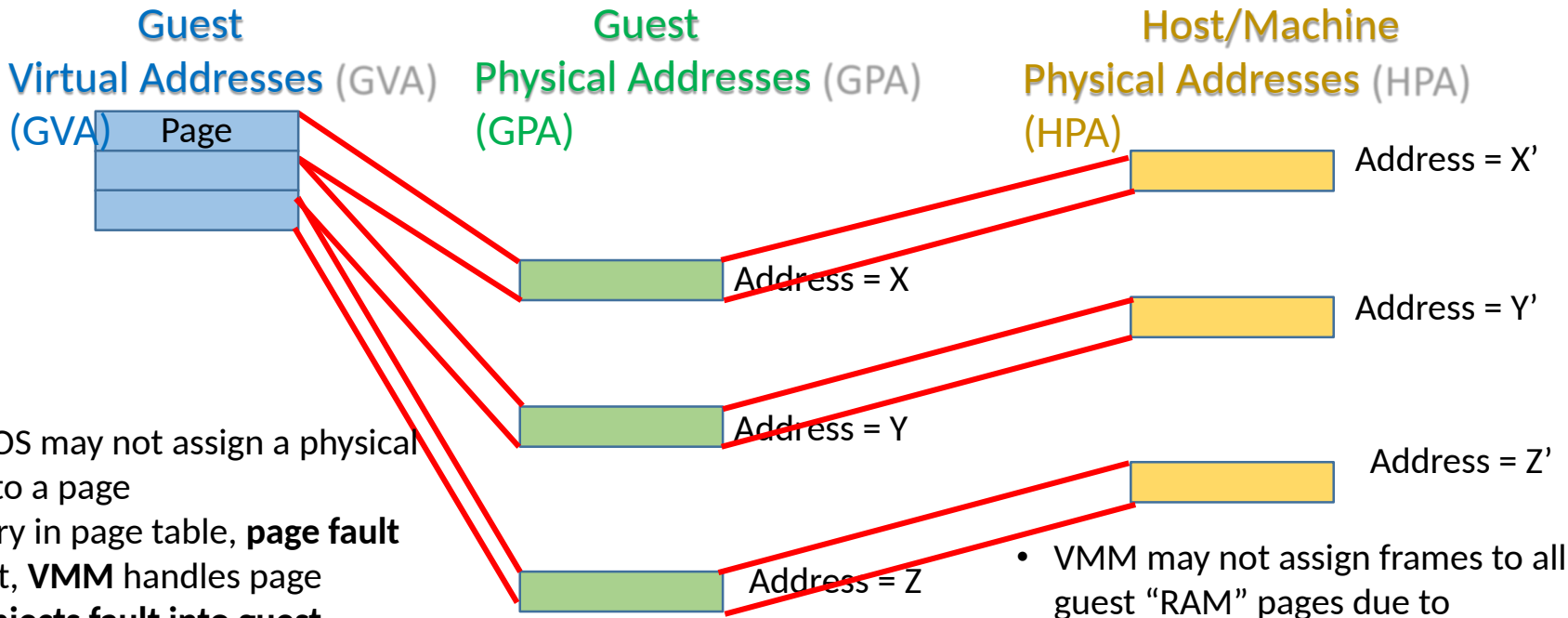
Shadow page tables ⇒ Software-based



Maintaining shadow page tables

- Guest writes to CR3, privileged operation traps to VMM
  - VMM marks the guest page table pages as read-only
  - VMM constructs shadow page table, sets CR3 to it
- Shadow page table can be built on demand
  - Start with empty page table, add entries on page faults
- Guest changes page table, traps to VMM, shadow entry updated
- Guest OS keeps multiple page tables of active processes in memory
  - On context switch, new page table used, but old page table still in memory
  - What about shadow page tables? How many in memory?
- Many design choices exist
  - VMM can discard old shadow page table on context switch, and rebuild it later (overhead during context switch)
  - VMM can maintain multiple shadow page tables of active processes (overhead to track changes to all page table pages)

Demand Paging and Page Faults



- Guest OS may not assign a physical frame to a page
- No entry in page table, **page fault**
- VM exit, **VMM** handles page fault, **injects fault into guest**
- **Guest assigns physical frame**

- VMM may not assign frames to all guest "RAM" pages due to memory pressure
- Page fault is handled by VMM
- Guest OS not aware of this fault
- "Hidden" page faults

Not part of syllabus

Memory reclamation technqiues

- VMM reclaims memory from guest “RAM” under memory pressure
- Uncooperative swapping: VMM reclaims some guest “RAM” pages and swaps them to disk
  - Page fault and memory assigned when guest accesses the page
  - May hurt performance, important pages may be swapped to disk
- Ballooning: VMM opens dummy device, requests pages from VM (“inflating the balloon”), then swaps these pages out
  - Since pages assigned to a device, pages not used by other processes in guest
  - Cooperative, guest can assign free pages
  - Lesser impact on performance, but reclamation takes time, requires guest changes
- Memory sharing: memory pages with identical content across VMs (OS images, zero pages etc.) shared between VMs
  - One physical frame mapped into page tables of multiple guests
  - Scan pages periodically to compute and match hash-based similarity of pages

I/O Virtualization

- Guest OS needs to access I/O devices, but cannot give full control of I/O to any one guest OS
- Two main techniques for I/O virtualization:
  - Emulation: guest OS I/O operations trap to VMM, emulated by doing I/O in VMM/host OS
  - Direct I/O or device passthrough: assign a slice of a device directly to each VM
- Many optimizations exist, active area of research

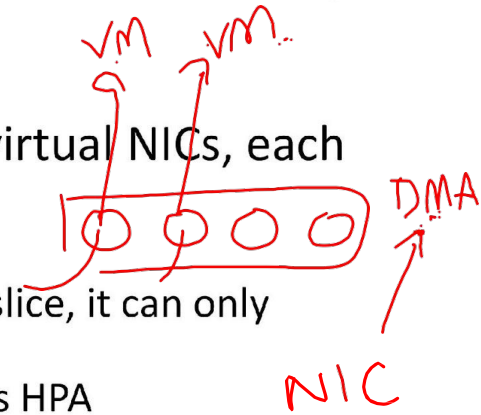
I/O Virtualization (1)



- Guest OS needs to access I/O devices, but cannot give full control of I/O to any one guest OS – how to perform I/O in guests?
- I/O Emulation: guest OS I/O operations trap to and emulated by VMM
  - Every time guest device driver gives command to I/O device, VM exit, VMM will issue command to I/O device on behalf of guest OS
  - Every time interrupt occurs, VM exit, VMM handles interrupt and injects it into guest VM it is destined for
  - I/O data DMA into host OS first, copied into guest later
  - Simple emulation slows I/O access down due to frequent VM exits, data copy
- Virtio: device drivers optimized for virtualization
  - Device driver batches I/O requests, exit once per batch of I/O requests
  - I/O data shared between guest and host via shared memory region, no copy

I/O Virtualization (2)

- Another way for guest OS to perform I/O: assign a slice of I/O device directly to each VM, each VM operates on its slice of device
  - Slices configured by VMM, does not compromise isolation of VMs
- Example: NICs with SR-IOV technology can create multiple virtual NICs, each slice is assigned to separate VM
  - Packet DMA directly into guest VM memory, host OS not involved
  - Challenge: when guest device driver provides DMA buffers to NIC slice, it can only provide guest physical addresses (GPA) of the buffer
  - NIC cannot access the DMA buffer memory using GPA alone, needs HPA
  - Solution: SR-IOV capable NICs have an inbuilt MMU (IOMMU) to translate from GPA to HPA
- Device passthrough techniques need hardware support, whereas virtio-based device drivers require only software upgrades for better performance



Summary

- Techniques for CPU virtualization
  - Paravirtualization: rewrite guest OS source code
  - Full virtualization: dynamic binary translation
  - Hardware-assisted virtualization: CPU has special virtualization mode
- Techniques for memory virtualization:
  - Shadow page tables: combined GVA→HPA mappings
  - Extended page tables: MMU is given separate GVA→GPA and GPA→HPA mappings
- I/O virtualization: emulation, device passthrough
- VMMs use a combination of above techniques
  - We will study all of the above techniques in detail